

大数据原理与技术 | 实验一

图像边缘检测与分类

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 王昌栋

起 始 日 期: 2026 年 3 月 16 日

结 束 日 期: 2026 年 3 月 17 日

实 验 地 点: B201

目录

1 实验要求 **2**

2 任务一 **2**

2.1 实现思路 2
2.2 实现代码 2
2.3 结果展示 3
2.4 注意事项 4

3 任务二 **5**

3.1 实验设计思路与数据集选取 5
3.2 核心代码展示 5
3.3 实验结果分析 6
3.4 注意事项 7

4 实验总结 **8**

1 实验要求

使用 Python+OpenCV 实现以下功能:

1. 任选一张图像，使用边缘检测算法 Sobel 检测图像边缘。展示原图像和分割后的图像。
2. 选择一种机器学习方法 (支持向量机 (SVM)、决策树等) 和一种深度学习方法 (MLP、CNN 等)，实现图像分类，数据集任选 (如 MNIST、CIFAR-10)，对比两种方法的准确率和计算效率。

2 任务一

2.1 实现思路

首先读取目标图像的灰度模式，利用 Sobel 算子分别计算水平 (X) 和垂直 (Y) 方向的梯度，将其绝对值化后并按比例合并，从而得到完整的边缘检测图像。随后，对合并后的梯度图像应用固定阈值进行二值化处理，实现目标的分割提取。

2.2 实现代码

任务一的完整代码如下：

```
import cv2
import os

def sobel_edge_detection_and_segmentation():
    #读取照片
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__
        )))
    img_path = os.path.join(base_dir, 'img', 'origin.png')

    #保存路径
    save_edge_path = os.path.join(base_dir, 'img', 'sobel_edges.png')
    save_seg_path = os.path.join(base_dir, 'img', 'segmented_result.png
        ')

    #读取图像
    gray_img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

    #计算 Sobel 梯度
    sobel_x = cv2.Sobel(gray_img, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(gray_img, cv2.CV_64F, 0, 1, ksize=3)

    #绝对值化并转回uint8(0-255标准图像格式)
    abs_sobel_x = cv2.convertScaleAbs(sobel_x)
    abs_sobel_y = cv2.convertScaleAbs(sobel_y)
```

```
23
24      # 边缘检测图
25      sobel_combined = cv2.addWeighted(abs_sobel_x, 0.5, abs_sobel_y,
            0.5, 0)
26
27      # 分割结果图
28      _, segmented_img = cv2.threshold(sobel_combined, 100, 255, cv2.
            THRESH_BINARY)
29
30      cv2.imwrite(save_edge_path, sobel_combined)
31      cv2.imwrite(save_seg_path, segmented_img)
32
33      cv2.imshow('1 - Original Grayscale', gray_img)
34      cv2.imshow('2 - Sobel Edges', sobel_combined)
35      cv2.imshow('3 - Segmented Image', segmented_img)
36
37      cv2.waitKey(0)
38      cv2.destroyAllWindows()
39
40  if __name__ == "__main__":
41      sobel_edge_detection_and_segmentation()
```

src/task1.py

2.3 结果展示

以下分别为原图、Sobel 边缘检测图和最终的分割结果图：

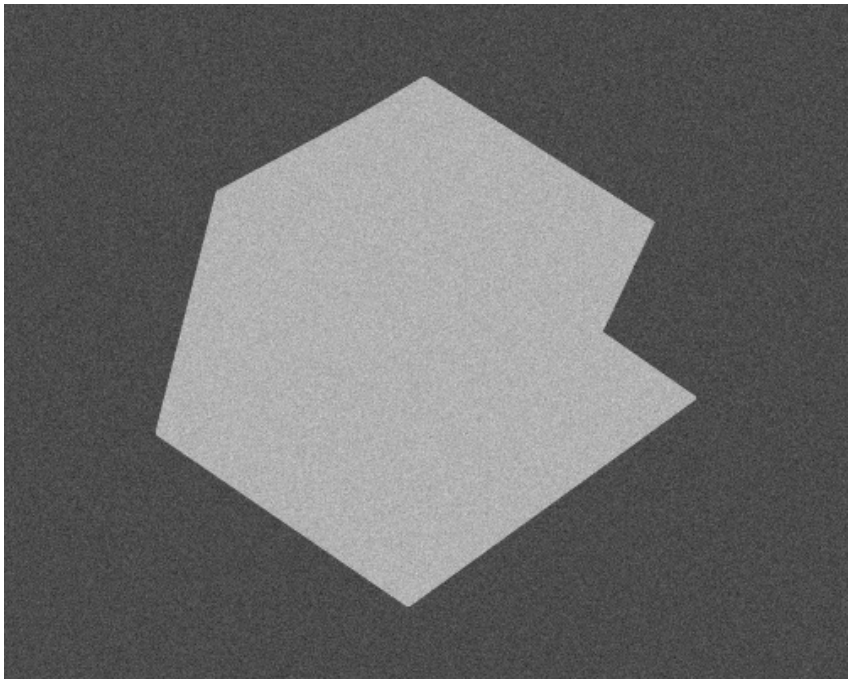


图 1: 原图像 (origin.png)

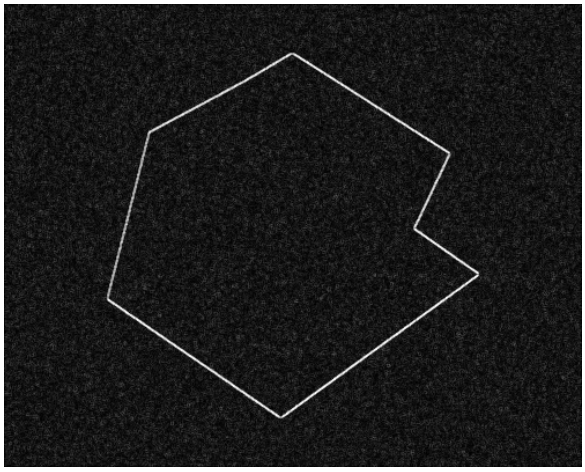


图 2: Sobel 边缘检测图像 (sobel_edges.png)

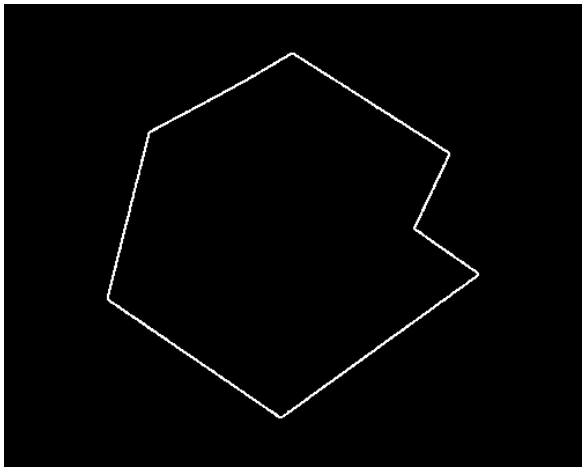


图 3: 分割后的处理图像 (segmented_result.png)

2.4 注意事项

1. 在计算 Sobel 梯度时，必须使用 `cv2.CV_64F` 数据类型，以防止出现负梯度被截断截零的情况，保证后续绝对值化时能保留完整的边缘信息。

2. 分割过程将梯度值大于设定阈值（例如 100）的像素设为 255 (纯白边缘)，小于设定阈值的设为 0 (纯黑背景)。100 这个阈值可根据 `origin.png` 的实际对比度进行微调，以达到最佳分割效果：

   - **阈值过小**：会出现明显的噪点，如图 4 所示，阈值为 50，结果出现噪点。

- **阈值过大**：可能导致连贯的边缘发生断裂，如图 5 所示，阈值为 200，出现边缘不完整的情况。
- **适当调整**：通过不断调阈值，观察分割结果，找到一个既能保留完整边缘又能有效抑制噪点的最佳阈值: 100。

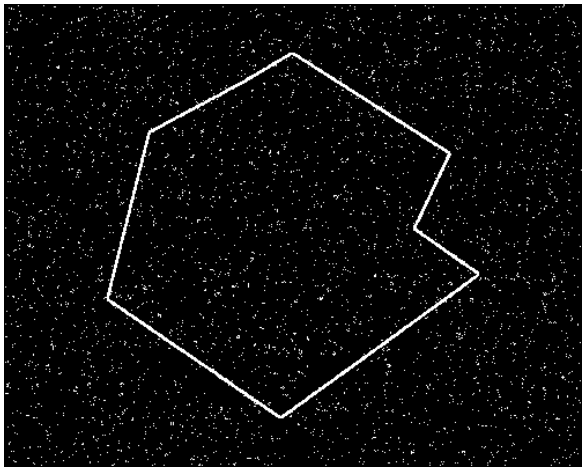


图 4: 阈值过小导致的噪点现象 (segmented_result1.png)

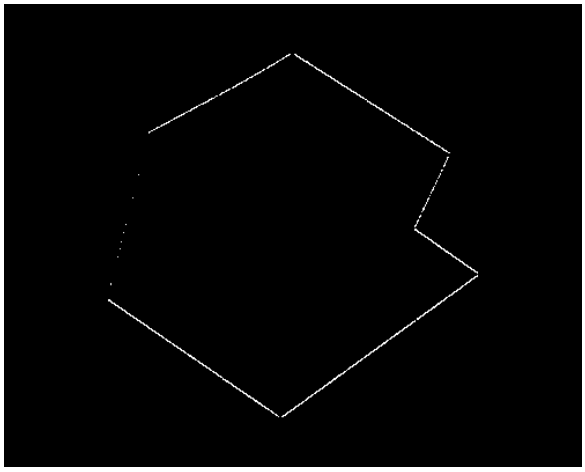


图 5: 阈值过大导致的边缘断裂现象 (segmented_result2.png)

3 任务二

3.1 实验设计思路与数据集选取

本实验旨在对比传统机器学习方法（支持向量机 SVM）与深度学习方法（卷积神经网络 CNN）在图像分类任务中的表现差异。

数据集选取：选取经典的 MNIST 手写数字数据集（28x28 单通道灰度图）。

SVM 模型设计：SVM 无法直接处理二维图像的空间拓扑结构，因此在预处理阶段，将 28x28 的图像展平为长度为 784 的一维向量，并进行归一化。为控制计算时间，只截取 10,000 张图片作为训练集，2,000 张作为测试集，采用 RBF 核函数进行训练。

CNN 模型设计：构建了包含一层卷积层、一层最大池化层和一层全连接层的轻量级网络。使用完整的 60,000 张训练集和 10,000 张测试集，通过交叉熵损失和 Adam 优化器进行迭代训练。

3.2 核心代码展示

1.SVM 数据降维与模型实例化：

```
#将二维图像转为一维特征向量
X_train_flat = X_train[:10000].reshape(10000, -1) / 255.0
X_test_flat = X_test[:2000].reshape(2000, -1) / 255.0

#RBF 核函数初始化并训练 SVM
svm_model = SVC(kernel='rbf', gamma='scale')
svm_model.fit(X_train_flat, y_train[:10000])
```

2.CNN 轻量化网络结构定义：

```python
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        #1通道输入，16通道输出，保留空间拓扑特征
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        #特征提取后进行全连接输出10个类别
        self.fc = nn.Linear(16 * 14 * 14, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

3.3 实验结果分析

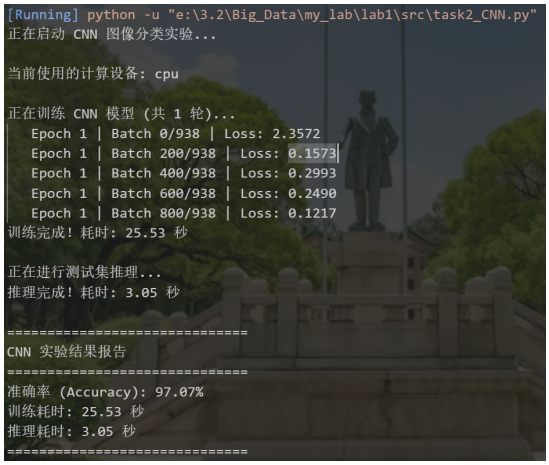


图 6: CNN(1 轮) 结果

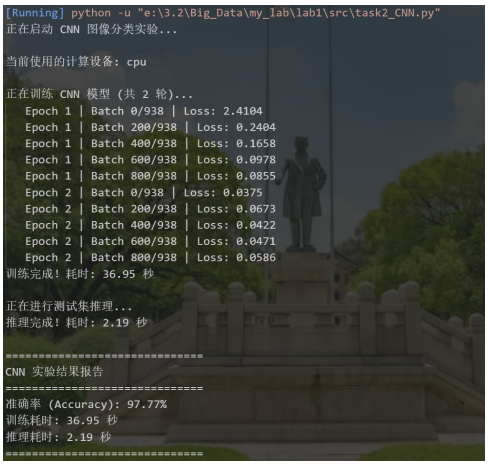


图 7: CNN(2 轮) 结果

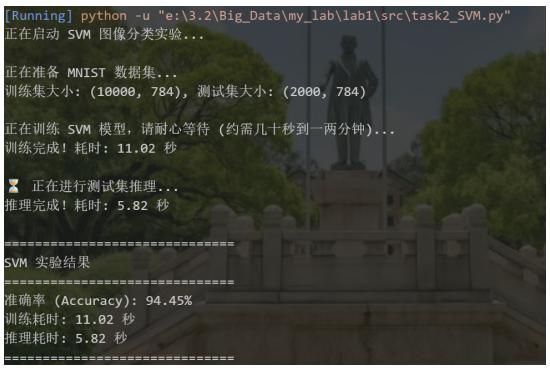


图 8: SVM 结果

根据控制台运行日志，SVM 与 CNN 在准确率和计算效率上的交叉对比数据如下：

模型方案	训练数据量	准确率 (Accuracy)	训练耗时	测试集推理耗时
SVM	10,000 张	94.45%	11.02 秒	5.82 秒 (2000 张)
CNN (1 Epoch)	60,000 张	97.07%	25.53 秒	3.05 秒 (10000 张)
CNN (2 Epochs)	60,000 张	97.77%	36.95 秒	2.19 秒 (10000 张)

1. 准确率对比分析

空间特征提取的降维打击：SVM 的准确率为 94.45%，这证明了其在简单特征上的有效性。然而，CNN 仅仅经过 1 轮完整训练（25.53 秒），准确率就直接跃升至 97.07%，并在第 2 轮达到 97.77%。根本原因在于 SVM 强行拉平图像，破坏了像素间的上下左右关系；而 CNN 通过卷积操作完美保留并提取了边缘、角点等二维空间特征。

2. 训练效率对比分析

算力扩展性差异：表面上看 SVM 训练（11.02 秒）比 CNN 稍快，但这建立在 SVM 仅处理了 1/6 数据的前提下。SVM 的时间复杂度随数据量呈非线性暴增，若强制 SVM 训练 6 万张图，耗时将达到数十分钟。相反，CNN 凭借“权值共享”机制，大幅减少了参数量，在处理海量图像数据时展现出了极强的延展性和效率。

3. 推理（预测）效率对比分析

巨大的部署优势：将测试集的耗时换算为单张图片的推理速度：

- SVM 单张推理耗时：5.82 秒 / 2000 张 $\approx$ 2.91 毫秒
- CNN 单张推理耗时：约 3 秒 / 10000 张 $\approx$ 0.30 毫秒

结论：CNN 的实际预测速度是 SVM 的近 10 倍。CNN 具备极大优势。

3.4 注意事项

纯 CPU 运行的有效性解读：本实验未调用 GPU。原因是本次设计的 CNN 模型结构极其轻量，且 MNIST 图像尺寸较小（28x28）。对于这种微小级别的网络矩阵运算，现代 CPU 的主频足以应对。若强制将小体量数据在 CPU 内存与 GPU 显存之间频繁搬运，其通信开销反而可能抵消 GPU 的并行计算优势。

4 实验总结

- **算法选择需与数据规模相匹配：**在小样本、特征明确的场景下，SVM 等传统模型依然具备极高的理论严谨性和可用性；但当数据量跃升至“大数据”级别（如完整的 6 万张图像）时，深度学习凭借权值共享等机制展现出了极强的数据延展性（Scalability）。工程实践中，必须根据数据量级（Data Scale）来权衡算法选型。
- **算力、精度与部署的权衡：**评价一个模型不能仅看最终的准确率（Accuracy）。CNN 在取得更高精度的同时，其单样本推理速度更是达到了 SVM 的十倍以上。在真实的大数据落地场景中，模型的推理延迟（Inference Latency）与吞吐量往往与模型的精度同等重要。
- **底层数据类型的溢出陷阱：**在使用 OpenCV 进行 Sobel 边缘检测时，若直接使用标准的 8 位无符号整数（`uint8`）存储梯度，会导致由明到暗的负梯度被截断截零，从而丢失半数边缘信息。

- **网络架构与硬件环境的适配：**实验证明，并非所有深度学习任务都必须过度依赖 GPU。对于架构极其轻量（如本实验的单层卷积与全连接）且数据集较小（MNIST）的任务，现代 CPU 依然能提供可观的训练与推理效率。